

UC: Estruturas de Dados e Análise de Algoritmos

Título: Prática nº 02C – Atividade – EDA2C_Lista – Complexidade Assintótica

Data: 26/05/2026

Integrantes:

- Sérgio Pinton Pavanelli - RA 123220202 (*líder: sergiopavanelli@gmail.com*)
- Samuel Zappala Batista - RA 12411504
- Elizabeth Stéphanhy Guimarães Miranda - RA 123220604
- Gabriel Victor Dornelas Ferreira Sathler - 12319216
- Ana Luiza Mattos de Carvalho - RA 124114111
- Júlia Starling Negrini Fudoli - RA 124222027
- Ana Clara Domingos Dias Silva - RA 12316965
- Miguel Pedro Pinheiro - RA 12315515
- Geovana Dias de Almeida - RA 123221311

Tema: Estrutura de Dados e Análise de Algoritmos

Objetivo: Complexidade Assintótica

Roteiro: Formar grupos e desenvolver as atividades propostas.

Entregas das atividades 1 e 2 solicitadas na lista de exercícios:

Atividade 1: Busca Linear em um array de inteiros

A busca linear percorre cada elemento do array sequencialmente, comparando-o com o valor alvo. Por não exigir nenhuma propriedade do array (como ordenação), é a estratégia mais geral e direta para a busca em coleções não ordenadas. O custo dessa generalidade é a eficiência: no pior caso é necessário inspecionar todos os n elementos.

(a) Algoritmo

Pseudocódigo:

```
algoritmo BuscaLinear(A, n, alvo)
    para i de 0 ate n - 1 faca
        se A[i] == alvo entao
            retorne i
    retorne -1
```

C++:

```
#include <iostream>
using namespace std;

int buscaLinear(int A[], int n, int alvo) {
    for (int i = 0; i < n; i++) {
        if (A[i] == alvo) {
            return i;
        }
    }
    return -1;
}

int main() {
    int n;
    cout << "Digite o tamanho do array: ";
    cin >> n;

    int A[100];
    cout << "Digite os " << n << " elementos do array:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "A[" << i << "]: ";
        cin >> A[i];
    }

    int alvo;
    cout << "Digite o valor alvo (target): ";
    cin >> alvo;

    int resultado = buscaLinear(A, n, alvo);

    if (resultado != -1) {
        cout << "Elemento encontrado no indice: " << resultado << endl;
    } else {
        cout << "-1 (elemento nao encontrado)" << endl;
    }

    return 0;
}
```

Exemplo de execução:

Array de entrada	Alvo	Saída
{3, 8, 1, 9, 4}	9	3
{3, 8, 1, 9, 4}	7	-1

(b) Análise de complexidade

A cada iteração do laço é executada **uma comparação** $A[i] == \text{alvo}$. O número de comparações depende de onde o alvo se encontra (ou se existe).

- **Melhor caso:** $T(n) = 1 = \Theta(1)$ — o alvo está em $A[0]$ e o laço encerra na primeira iteração.
- **Pior caso:** $T(n) = n = \Theta(n)$ — o alvo está em $A[n-1]$ ou não existe no array; todas as n comparações são realizadas.
- **Caso médio** (alvo presente com distribuição uniforme sobre os índices): cada posição tem probabilidade $1/n$ de conter o alvo, logo o número esperado de comparações é:

$$T_{\text{med}}(n) = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n)$$

Portanto, tanto no pior quanto no caso médio, a complexidade é $\Theta(n)$.

Conclusão: a busca linear é **ótima para arrays não ordenados** — qualquer algoritmo de busca em um array sem propriedade de ordenação precisa, no pior caso, examinar todos os n elementos (limite inferior $\Omega(n)$). Contudo, para grandes volumes de dados com arrays ordenados, algoritmos como a busca binária oferecem desempenho significativamente superior.

Atividade 2: Busca Binária em um vetor ordenado de inteiros

A busca binária explora a propriedade de **ordenação** do vetor para reduzir o espaço de busca pela metade a cada iteração. Compara-se o alvo com o elemento central do intervalo atual: se forem iguais a busca termina; se o alvo for menor, descarta-se a metade direita; se for maior, descarta-se a metade esquerda.

(a) Algoritmo e implementação em C++

Pseudocódigo (versão iterativa):

```
algoritmo BuscaBinaria(A, n, alvo)
    esq <- 0
    dir <- n - 1
    enquanto esq <= dir faça
        meio <- esq + (dir - esq) / 2    // evita overflow
        se A[meio] == alvo então
            retorne meio
        senao se A[meio] < alvo então
            esq <- meio + 1
        senao
            dir <- meio - 1
    retorne -1
```

C++:

```
#include <iostream>
using namespace std;

int buscaBinaria(int A[], int n, int alvo) {
```

```

int esq = 0;
int dir = n - 1;

while (esq <= dir) {
    int meio = esq + (dir - esq) / 2; // evita overflow de inteiro

    if (A[meio] == alvo) {
        return meio;
    } else if (A[meio] < alvo) {
        esq = meio + 1;
    } else {
        dir = meio - 1;
    }
}

return -1;
}

int main() {
    int n;
    cout << "Digite o tamanho do vetor (ordenado): ";
    cin >> n;

    int A[100];
    cout << "Digite os " << n << " elementos em ordem crescente:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "A[" << i << "]: ";
        cin >> A[i];
    }

    int alvo;
    cout << "Digite o valor alvo (target): ";
    cin >> alvo;

    int resultado = buscaBinaria(A, n, alvo);

    if (resultado != -1) {
        cout << "Elemento encontrado no indice: " << resultado << endl;
    } else {
        cout << "-1 (elemento nao encontrado)" << endl;
    }

    return 0;
}

```

Exemplo de execução:

Array de entrada (ordenado)	Alvo	Saída
{1, 4, 7, 10, 15, 22, 30} (n=7)	15	4
{1, 4, 7, 10, 15, 22, 30} (n=7)	5	-1

Pré-requisito: o vetor deve estar **ordenado** antes de chamar a busca binária. Caso não esteja, o resultado é indefinido. Para vetores não ordenados, utilize a busca linear (Atividade 1) ou ordene o vetor previamente (ex.: `std::sort`, com custo adicional de $O(n \log n)$).

(b) Análise da complexidade assintótica — demonstração de $O(\log n)$

Intuição geométrica da divisão pela metade:

A cada iteração do laço `while`, o intervalo de busca `[esq, dir]` tem seu tamanho reduzido pela metade:

Iteração	Tamanho do intervalo
0	n
1	$n / 2$
2	$n / 4$
k	$n / 2^k$

O laço encerra quando o intervalo se torna vazio, isto é, quando restar **1 elemento** ou menos:

$$\frac{n}{2^k} \leq 1 \implies 2^k \geq n \implies k \geq \log_2 n$$

Portanto o número máximo de iterações é $k = \lceil \log_2 n \rceil$, e cada iteração executa $O(1)$ operações (cálculo do meio + uma comparação + ajuste de `esq` ou `dir`).

Relação de recorrência (versão recursiva equivalente):

```
T(1) = O(1)           // caso base: intervalo de 1 elemento
T(n) = T(n/2) + O(1)  // divide pela metade + trabalho constante
```

Aplicando o **Teorema Mestre** com $a = 1$, $b = 2$, $f(n) = O(1)$:

- $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = O(1)$
- Como $f(n) = O(n^{\log_b a}) = O(1)$, estamos no **Caso 2** do Teorema Mestre.
- Resultado: $T(n) = O(\log n)$.

Casos de complexidade:

- Melhor caso:** $T(n) = O(1)$ — o alvo coincide com o elemento central logo na primeira comparação.
- Pior caso:** $T(n) = \lceil \log_2 n \rceil + 1 = O(\log n)$ — o alvo está nas extremidades do vetor ou não existe; o intervalo é dividido ao máximo.
- Caso médio:** $T(n) = O(\log n)$ — a análise esperada produz o mesmo crescimento logarítmico.

Exemplo numérico:

n (tamanho do vetor)	Busca Linear — comparações (pior caso)	Busca Binária — comparações (pior caso)
10	10	4

n (tamanho do vetor)	Busca Linear — comparações (pior caso)	Busca Binária — comparações (pior caso)
100	100	7
1.000	1.000	10
1.000.000	1.000.000	20
1.000.000.000	1.000.000.000	30

O crescimento logarítmico é praticamente constante comparado ao linear: para $n = 10^9$, a busca binária executa apenas **~30 comparações**.

Conclusão: a busca binária é **ótima em ordem assintótica** para o problema de busca em vetores ordenados — seu limite inferior de complexidade também é $\Omega(\log n)$, pois qualquer algoritmo de comparação precisa distinguir entre n posições possíveis com respostas binárias, o que requer ao menos $\log_2 n$ comparações. A busca binária atinge exatamente esse limite, tornando-se a solução mais eficiente para arrays estáticos ordenados.

Comparativo Final

Característica	Busca Linear	Busca Binária
Pré-requisito	Nenhum	Vetor ordenado
Melhor caso	$\theta(1)$	$\theta(1)$
Pior caso	$\theta(n)$	$\theta(\log n)$
Caso médio	$\theta(n)$	$\theta(\log n)$
Escalabilidade	Ruim (cresce linearmente)	Excelente (quase estável)
Simplicidade	Alta	Média
Aplicação indicada	Arrays pequenos/não ord.	Arrays grandes ordenados

A análise assintótica demonstra que a escolha do algoritmo correto tem impacto crítico no desempenho real de sistemas: enquanto a busca linear é suficiente para pequenos volumes, a busca binária se torna indispensável à medida que o tamanho dos dados cresce.
